

El Escenario: Debes crear un sistema para gestionar proyectos y empleados. La base de datos debe manejar acentos, los empleados deben tener sueldos controlados, y los usuarios del sistema deben estar segmentados por roles de lectura o administración.

Paso 1: Definición de Base de Datos y Tablas (DDL)

- **Crea la BD gestion_proyectos con cotejamiento español.**

Crear la Base de Datos Usamos CHARACTER SET y COLLATE para asegurar que las ñ y tildes no den problemas.

```
CREATE DATABASE gestion_proyectos
```

```
CHARACTER SET utf8mb4 COLLATE utf8mb4_spanish_ci;
```

```
USE gestion_proyectos;
```

- **Crea la tabla departamentos (id autoincremental, nombre único).**

Tabla Departamentos (Padre) Creamos primero la tabla "padre" porque la tabla de empleados dependerá de ella.

AUTO_INCREMENT: Para que el ID se genere solo (1, 2, 3...).

UNIQUE: Para que no existan dos departamentos con el mismo nombre.

```
CREATE TABLE departamentos (  
  id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(50) NOT NULL UNIQUE  
);
```

Crea la tabla empleados con:

id_empleado (PK).

dni (Texto de 9, no nulo).

sueldo (Decimal, por defecto 1200).

fk_depto (FK hacia departamentos).

Tabla Empleados (Hija) Aquí aplicamos las restricciones fuertes.

DEFAULT 1200: Si no especificamos sueldo, se guarda 1200 automáticamente.

CHECK: "El portero". Impide guardar sueldos negativos.

FOREIGN KEY ... ON DELETE SET NULL: La regla de seguridad. Si borramos un departamento, no despedimos al empleado; simplemente su campo departamento se queda vacío (NULL).

```
CREATE TABLE empleados (  
  id_empleado INT NOT NULL PRIMARY KEY,  
  dni VARCHAR(9) NOT NULL,  
  sueldo DECIMAL(10,2) DEFAULT 1200,  
  fk_depto INT,
```

-- Restricción de validación de datos

```
CONSTRAINT chk_sueldo_valido CHECK (sueldo > 0),
```

```
-- Relación con la tabla padre
```

```
CONSTRAINT fk_empl_dept
```

```
FOREIGN KEY (fk_depto) REFERENCES departamentos(id)
```

```
ON DELETE SET NULL
```

```
);
```

Modificación: Añade una columna email a la tabla empleados después de crearla.

Modificación de estructura Se nos olvidó el email, así que alteramos la tabla ya creada.

```
ALTER TABLE empleados
```

```
ADD COLUMN email VARCHAR(100) AFTER dni;
```

Paso 2: Manipulación de Datos (DML)

Manipulación de Datos (DML)

Ahora que tenemos el esqueleto, metemos información.

Inserta un departamento llamado 'I+D'.

Insertar datos Primero el departamento (para que exista el ID 1) y luego el empleado asignado a ese ID.

-- Insertamos Dept 'I+D'. Como es auto_increment, se le asignará el ID 1.

```
INSERT INTO departamentos (nombre) VALUES ('I+D');
```

Inserta un empleado en ese departamento.

-- Insertamos empleado asignándolo al Dept 1.

```
INSERT INTO empleados (id_empleado, dni, email, sueldo, fk_depto)
VALUES (101, '12345678Z', 'ana@test.com', 1500.50, 1);
```

Sube el sueldo un 10% a todos los empleados.

Actualizar datos (Cálculos) Subimos el sueldo. Al no poner WHERE, afecta a toda la tabla.

```
UPDATE empleados
```

```
SET sueldo = sueldo * 1.10;
```

Borra los departamentos que no tengan nombre.

Borrar datos Borrarnos los departamentos sin nombre (aunque tengan restricción NOT NULL, esta es la sintaxis correcta para buscar vacíos).

```
DELETE FROM departamentos
```

```
WHERE nombre IS NULL;
```

Paso 3: Usuarios, Permisos y Roles (DCL)

Gestión de Roles Creamos los "sombreros" (roles) y definimos qué poderes tiene cada sombrero.

Crea un Rol de "Solo Lectura".

-- Creación de roles

```
CREATE ROLE 'rol_lectura';
```

-- Asignación de permisos a los roles

-- Lectura: Solo puede hacer SELECT en todas las tablas de esta BD

```
GRANT SELECT ON gestion_proyectos.* TO 'rol_lectura';
```

Crea un Rol de "Escritura" que pueda hacer de todo en esa BD.

```
CREATE ROLE 'rol_escritura';
```

-- Escritura: Puede hacer TODO (ALL PRIVILEGES) en esta BD

```
GRANT ALL PRIVILEGES ON gestion_proyectos.* TO 'rol_escritura';
```

Crea un Usuario analista_datos.

Gestión de Usuarios Creamos la persona real.

```
CREATE USER 'analista_datos'@'localhost' IDENTIFIED BY 'password1234';
```

Asigna el rol de lectura al usuario y luego quítaselo (REVOKE) para darle el de escritura.

Asignación y cambio de roles Primero le damos el sombrero de lectura. Luego se lo quitamos (REVOKE) para darle el de jefe (GRANT).

-- Asignar rol inicial

```
GRANT 'rol_lectura' TO 'analista_datos'@'localhost';
```

-- Cambiar de rol

```
REVOKE 'rol_lectura' FROM 'analista_datos'@'localhost';
```

```
GRANT 'rol_escritura' TO 'analista_datos'@'localhost';
```

1. PRIMARY KEY (La Matrícula Única)

En el código: `id INT PRIMARY KEY`

- **¿Qué es?** Es el **identificador único e irrepetible** de cada fila. Es como el DNI de una persona o la matrícula de un coche.
- **Reglas de oro:**
 1. No puede haber dos iguales (no pueden existir dos empleados con ID 101).
 2. No puede estar vacío (NULL).
- **Para el examen:** Siempre tienes que poner una. Si no hay un dato natural (como el DNI), te inventas un `id` numérico.

2. AUTO_INCREMENT (El Contador Automático)

En el código: `id INT AUTO_INCREMENT`

- **¿Qué es?** Es una comodidad. Le dices a la base de datos: *"Yo no quiero preocuparme de qué número toca ahora. Si el último fue el 5, ponle al siguiente el 6"*.
- **¿Por qué se usa?** Para evitar errores. Imagina que tienes 1000 empleados y tienes que meter al 1001. ¿Te vas a poner a mirar cuál fue el último? Mejor que lo haga la máquina.

3. NOT NULL (Prohibido Dejar en Blanco)

En el código: `nombre VARCHAR(50) NOT NULL`

- **¿Qué es?** Haces obligatorio rellenar ese campo.
- **Ejemplo:** En un formulario de registro, el campo "Email" suele ser `NOT NULL` (obligatorio), pero "Segundo Apellido" puede ser `NULL` (opcional).
- **En tu examen:** El DNI es `NOT NULL` porque un empleado sin DNI no puede existir legalmente.

4. UNIQUE (Sin Dobles)

En el código: `nombre VARCHAR(50) UNIQUE`

- **¿Qué es?** Garantiza que no se repita el valor, pero (a diferencia de la Primary Key) no es el identificador principal de la tabla.
- **Diferencia con Primary Key:**
 - Una tabla solo tiene **UNA** Primary Key.
 - Pero puede tener muchos campos `UNIQUE`.
- **Ejemplo:** Tu ID de empleado es la Primary Key. Tu email corporativo es `UNIQUE` (nadie más puede tenerlo), pero no es tu ID.

5. FOREIGN KEY o Clave Foránea (El Puente)

En el código: `CONSTRAINT fk_... FOREIGN KEY (fk_depto) REFERENCES departamentos(id)`

- **¿Qué es?** Es el pegamento que une dos tablas.

- En la tabla *Empleados*, tienes una columna `fk_depto`.
- Esa columna apunta (hace referencia) al `id` de la tabla *Departamentos*.
- **¿Para qué sirve?** Para mantener la **integridad**.
 - Si intentas crear un empleado asignado al departamento 99, y el departamento 99 NO existe en la tabla de departamentos, MySQL **te dará error** y no te dejará guardar al empleado. ¡Te protege de datos basura!

6. ON DELETE SET NULL (El Plan de Emergencia)

En el código: `ON DELETE SET NULL`

- **¿Qué es?** Es la instrucción de qué hacer si alguien borra el dato original.
- **El escenario:** Imagina que cierras el departamento de "Ventas" (lo borras). ¿Qué pasa con Juan, que trabajaba allí?
 - **Si usas CASCADE:** Despiden a Juan (se borra el empleado). ¡Peligroso!
 - **Si usas SET NULL:** Juan se queda en la empresa, pero su campo "departamento" se queda vacío (NULL). Se queda "en el limbo" hasta que lo reasignes. Esta es la opción segura que hemos usado.

7. CHECK (El Portero de Discoteca)

En el código: `CHECK (sueldo > 0)`

- **¿Qué es?** Es una regla de validación. Antes de guardar un dato, MySQL comprueba si cumple esta condición.
- **Ejemplo:** `CHECK (edad >= 18)`. Si intentas meter a alguien de 15 años, MySQL te dice: "¡Eh! Aquí no entras".
- **En tu examen:** `CHECK (sueldo > 0)` impide que por error le pongas un sueldo negativo a alguien.

8. DEFAULT (El Valor por Defecto)

En el código: `DEFAULT 1200`

- **¿Qué es?** Es el valor "por si acaso". Si al insertar un empleado no especificas cuánto cobra, MySQL le pondrá 1200 automáticamente en lugar de dejarlo vacío o dar error.